P6P9 LLD

Table of Contents

- Summary
- Business purpose
- Core Business Functions
- Service Catalog
- Batch Specific Information and Business Logic
- Storage Architecture
- Sequence Diagrams
- Technologies
- API
- UI
- 1. Non-Prod
- 1.1 Private End Point
- 1.2 Tags
- 1.3 User Access
- 1.4 Service Principal
- 2. Prod
- 2.1 Private End Point
- 2.2 Tags
- 2.3 Service Principal
- 2.4 User Access
- Github & Pipeline
- PV Groups
- Entra AD Registration
- Data Mapping and Schema Design
- Rollback Plan
- Monitoring and Logging
- Validation Checkpoints
- Final Design Position

Summary

P6P9 is a batch-based payroll tax-code processing service, modernized from a legacy system and deployed as a Spring Boot application on AKS. It operates as a background, non-customer-facing component and follows a file-driven model where inbound CSV files are received via Azure File Share, processed using Spring Batch, and archived after execution. During processing, each record is validated, checked for policy eligibility and updated with new tax codes via the clanad-policy-api. The design separates responsibilities by delegating policy validation and tax updates to dedicated APIs. Processed results are written to structured success and error archives to ensure traceability, and API versioning is managed through the Accept header to support backward compatibility and controlled evolution.

Business purpose

P6p9 modernization provides a controlled spring boot batch service capability for payroll tax-code files in the LV Clanad.

Its purpose is to:

- receive payroll tax-code input files from an internal file share
- validate each row against policy data
- apply valid tax-code updates through modern services
- to generate success and failure output files
- run as a scheduled or controlled on-demand batch

Core Business Functions

Functions	Sub-function	Functionalities
File Intake	Watched Folder	Scans app.input-dir on startup for *.csv files (sorted) and processes them sequentially using BatchOrchestrator.
	Header Handling	Treats the first row as header and propagates it to success archive outputs via RowSplitArchiveListener.
Record Model	P45 CSV Mapping	Maps fixed CSV columns to P45CsvRecord including tax year, NI, employer, PAYE form type, tax code, indicators, previous pay/tax, and effective date.
Validation	Field Validation	Validates NI format (NiFormatter), PAYE code syntax (TaxCodeValidator), and effective date ≤ current date; supports multiple date formats.
	Data Normalisation	Normalises input data and converts previous tax values to negative where required (legacy-compatible behaviour).
PAYE Code Processing	Code Assembly	Builds PAYE code by combining taxRegime and taxCode; appends * for emergency basis (weekMonthIndicator = X).
Policy Validation	Eligibility Check	Calls Policy API (GET /api/policy/tax-code-eligibility) with required identifiers using versioned Accept header.
	Response Handling	Processes API response; rejects ineligible records with reasons and adjusts values where required (e.g., zero previous pay/tax).
Tax Code Processing	Tax Update	Calls Tax API (POST /api/policy/tax-code-updates) to apply validated tax-code updates.
	Response	Interprets API responses to determine success or failure of each record.

	Handling	
Persistence (Domain)	API-driven Updates	Delegates all business data updates to external APIs; no direct database writes in the main processing flow.
Persistence (Technical)	Batch Metadata	Uses PostgreSQL for Spring Batch job repository and execution tracking.
Result Handling	Success Output	Writes successfully processed records to archive/success/<filename>-<timestamp>.csv.
	Failure Output	Writes failed or skipped records to archive/failure/<filename>-unmatched.csv with error details.
	File Lifecycle	Deletes input file on successful completion; moves file to error archive on failure (FileArchiveService).
Batch Processing	Chunk Processing	Processes records in configurable chunks (app.chunk-size, default 100) with transactional boundaries.
	Skip Handling	Skips recoverable exceptions (P45ProcessingException, FlatFileParseException) without failing the entire job.
	Restartability	Supports restart using JobParameter (filePath) and resumes from last successful commit.
Integration Resilience	Retry Logic	Retries transient API failures (5xx, network issues) using exponential backoff; does not retry 4xx errors.
Security	API Access	Supports outbound authentication using Authorization: Bearer <api.auth-token> for Policy and Tax APIs.

Service Catalog

Service	Category	Endpoint Path	HTTP Method	Operation Type	Primary Database Operation	Schema	Primary Table	Secondary Tables	Description
Policy Eligibility API	Policy Validation	/api/policy/tax-code-eligibility	GET	SELECT	Read policy eligibility data	policy_service	MASTER_ARCHIVE	—	Validates whether a policy is eligible for tax-code update based on NI, works number, tax year, PAYE form type, and employer reference.
Tax Code Update API	Tax Processing	/api/policy/tax-code-updates	POST	INSERT / UPDATE	Update tax code records	policy_service	PAY_HEADER	PAYEHIST, PAPAYHIS	Applies tax-code updates for eligible policies and persists

									updates in downstream systems.

Target service interaction:

1. P6p9-batch reads the file
2. P6p9-batch calls clanad-policy-api for policy eligibility and tax-code update
3. P6p9-batch writes success and failure files.

Batch Specific Information and Business Logic

- ## Batch Input and Execution Model

Item	Design
Input Type	CSV files (P6/P9/P45) from Azure File Share
Execution Style	Scheduled (AKS CronJob) with optional on-demand run
Processing Unit	Chunk-based processing (Spring Batch, default ~100 records per transaction)
Output Artifacts	Success file, error file, archived input file, batch metadata
Failure Handling	Invalid rows are skipped and written to error file; technical failures may stop job; no full file rollback
Restartability	Job restarts from last successful chunk using filePath
File Lifecycle	File deleted on success; moved to error archive on failure

Legacy CSV Columns Carried Forward

The legacy implementation requires at least 15 columns and uses the following positions:

Column Index	Meaning
6	NI number
7	Works or payroll number
10	Tax code
11	Week1 flag (`x` becomes `*`)
12	Tax regime
13	Previous pay
14	Previous tax
15	Effective date

Batch Business Rules

Rule	Description
Minimum column count	Reject row if fewer than 15 columns are present
Policy number parsing	Support normal policy number, .1 , and /1 point-one forms
NI validation	Reject row if NI number is blank
Tax code construction	Build tax code from regime and code, with X mapped to *
Tax code validation	Accept BR , NT , D0 , D1 , valid K values, and valid suffix forms reflected by the legacy logic
Effective date rule	Reject row if effective date is greater than current date
Policy lookup	Validate against the MASTER_ARCHIVE equivalent in the policy service
Policy status rule	Accept only L or P ; reject Q as migrated and reject all other statuses
P45 tax-year rule	If current tax year equals the tax year of P45_RECEIVED , set previous pay and previous tax to zero before update
Output rule	Accepted rows go to success output; rejected rows go to failure output with reason

- ### Modern Batch Output Design

Output	Purpose
Success File	Contains records successfully processed and updated via Policy API; generated at step completion and written to archive/success/<filename>-timestamp.csv.
Failure File	Contains records rejected due to validation errors, policy ineligibility, or API failures; includes rejection reason and written to archive/failure/<filename>-unmatched.csv.
Archived Input File	Original input file preserved after processing; deleted on success or moved to failure archive on failure (FileArchiveService).
Batch Run Record	Stores job-level execution details (status, start/end time, step execution) in Spring Batch metadata tables (BATCH_JOB_EXECUTION).
Batch Row Result Record	Row-level results are not stored in database tables; instead captured in success/failure output files with processing outcome and reason.

Processing Logic

1. Validate and normalise input record (NI, tax code, dates).

2. Validate policy eligibility via clanad-policy-api.
3. Apply tax-code update via clanad-policy-api for eligible records.
4. Capture processing outcome (success or failure) for each record.
5. Route records to appropriate output (success or failure file with reason).

Retry Strategy

Parameter	Value
Max Retries	3
Retry Backoff	Exponential
Retryable Errors	Network Failure, HTTP 500

Retry pattern:

- Attempt 1: immediate
- Attempt 2: after 5 seconds
- Attempt 3: after 20 seconds

Idempotency

Each row is uniquely identified by:

`file_name + row_number`

This key prevents duplicate tax updates when retries or batch restarts occur.

Storage Architecture

Storage Account Proposal

Property	Value
Storage Account	st-clanad-p6p9-batch
Region	UK South
Performance	Standard
Replication	LRS
Encryption	Azure Storage Encryption
Network Access	Private Endpoint Only

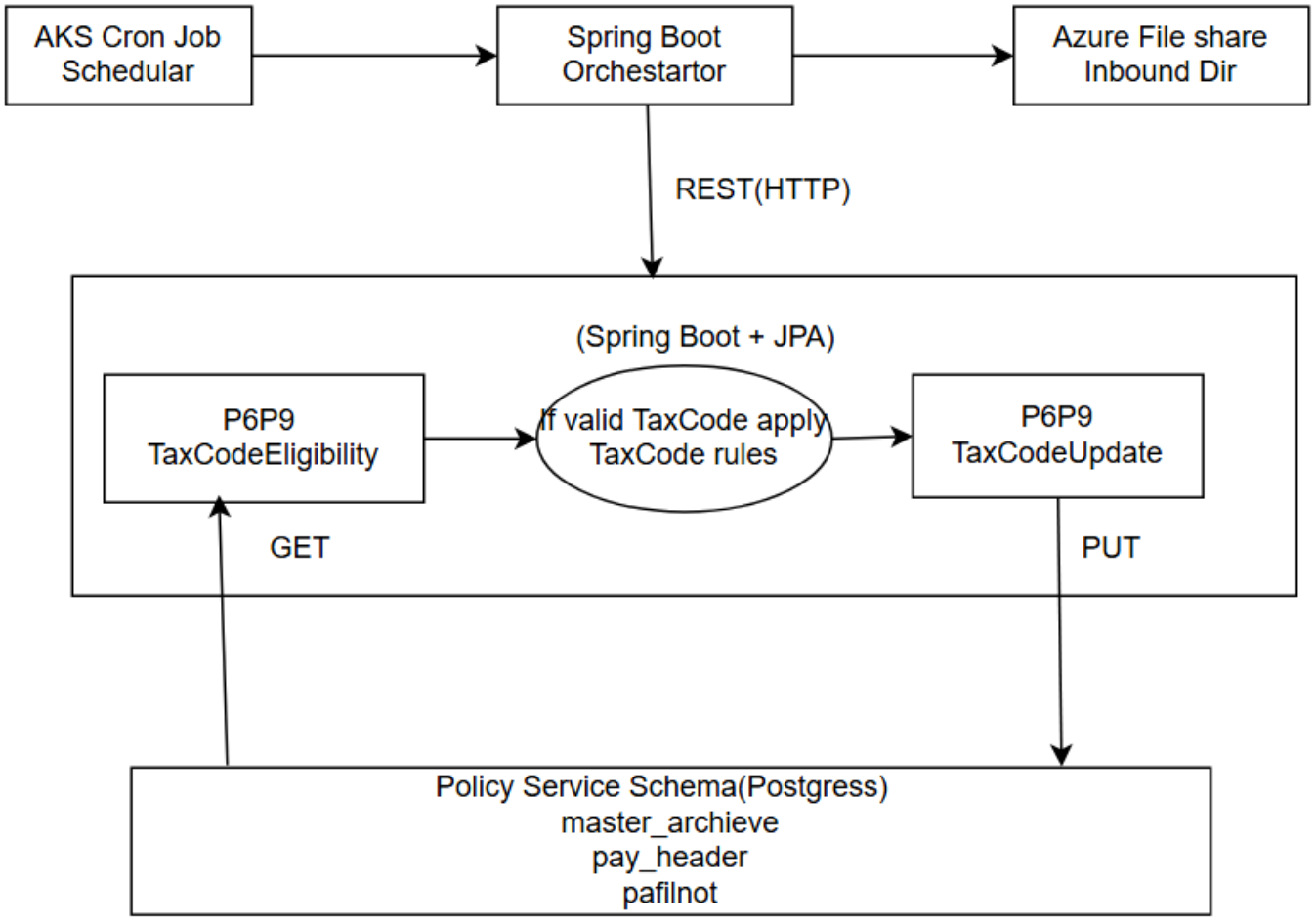
Public Access	Disabled

Folder Structure

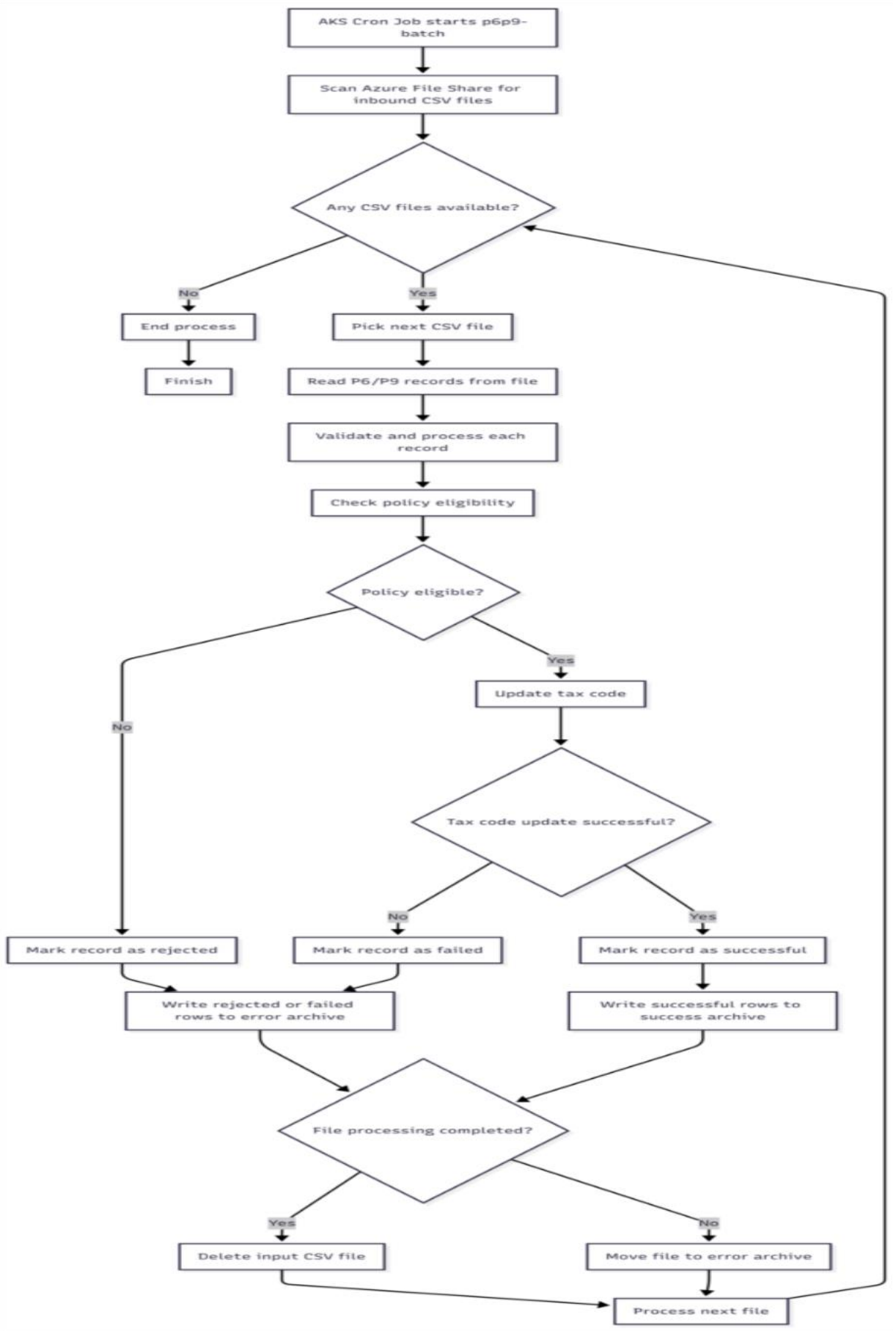
/p6p9
/input
/archive
/success
/failure

Folder	Purpose
/input	Stores incoming CSV files (P6/P9/P45) to be processed by the batch job.
/archive/success	Contains successfully processed output files (success records) and completed input files (if retained).
/archive/failure	Contains failed or rejected records and input files when processing fails.

Architecture Diagram

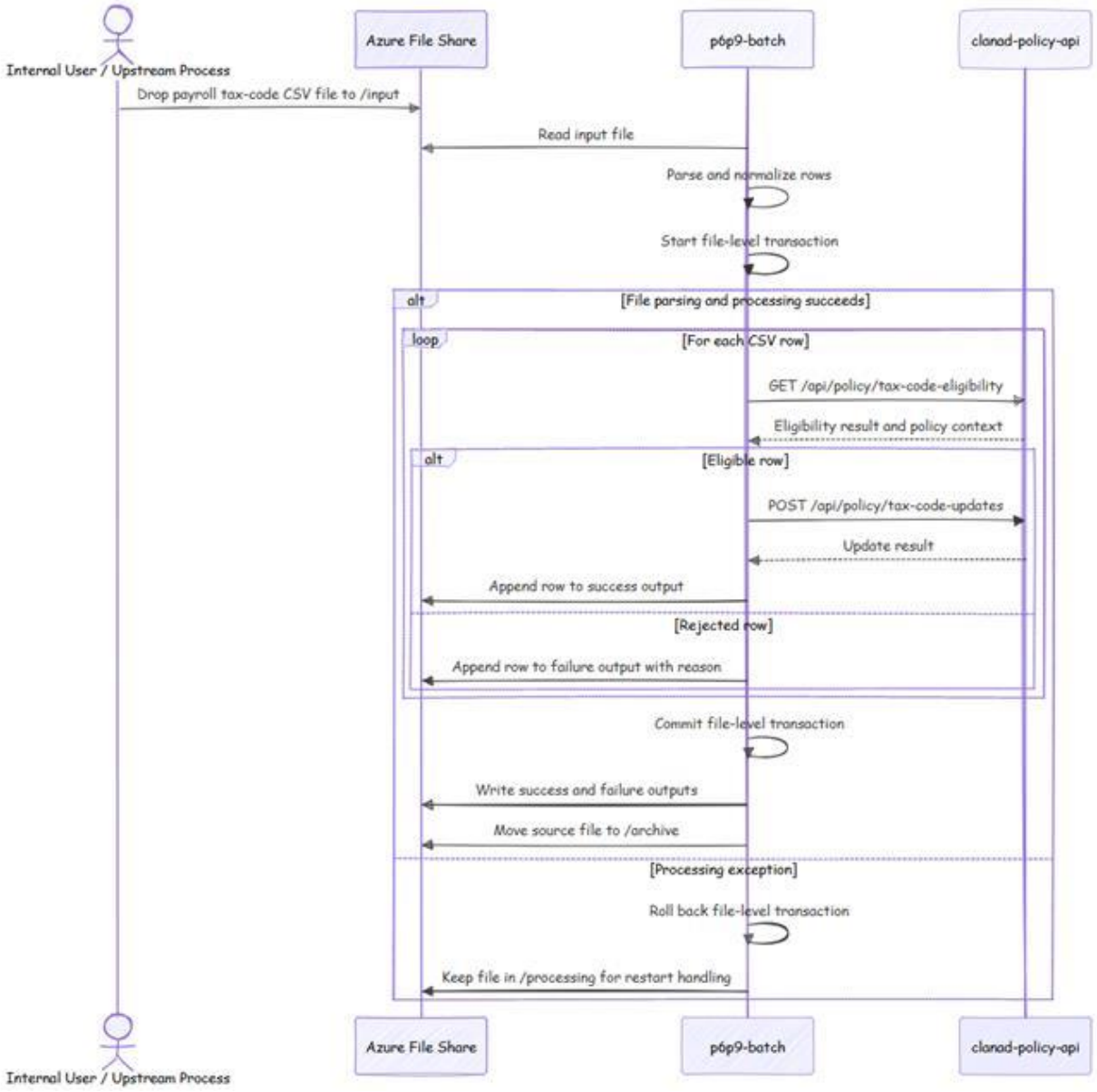


Business process flow Diagram



Sequence Diagrams

End-to-End Batch Flow



Sequence meaning:

1. The source file is placed in the /input folder of the private file share.
2. P6p9-batch reads and parses the file and starts file-level transactional processing.
3. Each row is validated through the policy service.
4. Valid rows are sent to the tax service for update, while rejected rows are written to the failure output.
5. If the file finishes successfully, the file-level transaction is committed, success and failure outputs are finalized, and the source file is moved to /archive.
6. If a processing exception occurs, the file-level transaction is rolled back
7. If the batch crashes, the file remains in /input so the restarted batch can resume controlled handling.
8. This reflects the legacy behavior more accurately: rollback is file-level on exception, not a separate row-level business rollback for each rejected record.

Technologies

Category	Technology
Language	Java 25
Framework	Spring Boot 4.x
Web	Spring Web
Security	Spring Security
ORM	Spring Data JPA
Database	PostgreSQL 16
Documentation	SpringDoc OpenAPI
Health and Metrics	Spring Boot Actuator
Testing	JUnit, Mockito, Testcontainers
Deployment	Docker on AKS

Deployment model:

- P6p9-batch deployed as Kubernetes CronJob, with optional on-demand
- Clanad-policy-api as always-running service
- Clanad-policy-api as always-running service

API

The P6P9 batch does not expose a user-facing API. It operates as a background batch service, scheduled via platform tooling (AKS CronJob) or triggered on demand.

Only the required downstream service interfaces are defined.

Required service interfaces

clanad-policy-api

Endpoint

GET /api/policy/tax-code-eligibility

Purpose

Validates the inbound record against policy data, determines eligibility for tax-code update, handles policy number normalisation (including point-one cases), and provides policy context required for further processing.

Request definition

Field	Type	Required	Description
niNumber	string	Yes	NI number from the inbound file
payrollNumber	string	Yes	Works or payroll number from the inbound file
taxYear	string	Yes	Tax year from inbound record
payeFormType	string	Yes	P6 / P9/ P45 indicator
employerReference	string	Yes	Employer reference from inbound file

Response definition

Field	Type	Description
eligible	boolean	Indicates if the record can proceed to tax update
normalizedPolicyNumber	number	Policy number after normalisation (including point-one handling)
pointOnePolicy	boolean	Indicates if point-one matching was applied
policyStatus	string	Current policy status used for validation
p45Received	date	P45 received date used for tax-year rule
rejectionReasons	array[string]	List of business validation failure reasons

Clanad-policy-api

Endpoint

POST /api/policy/tax-code-updates

Purpose

Apply the validated tax-code update command that replaces the legacy **UPDATE_TAXCODE** stored-procedure boundary.

Request definition

Field	Type	Required	Description
policyNumber	number	Yes	Normalized policy number returned from eligibility validation
taxCode	string	Yes	Final tax code derived from the inbound file
previousPay	number	Yes	Previous pay amount after any P45 same-tax-year adjustment
previousTax	number	Yes	Previous tax amount after any P45 same-tax-year adjustment
effectiveDate	date	Yes	Source effective date retained for audit
sourceFileName	string	Yes	Source file name for audit

Response definition

Field	Type	Description
updated	boolean	True when the tax-code update is committed successfully
resultCode	string	Outcome code for support and monitoring
resultMessage	string	Human-readable outcome or failure message
auditReference	string	Audit or tracking reference for the update

API request examples

Policy eligibility request

```
{
  "niNumber": "QQ123456C",
  "payrollNumber": "WRK001234",
  "taxYear": "2025",
  "payeFormType": "P45",
  "employerReference": "123/AB456"
}
```

Tax-code update request


```
{
  "policyNumber": 987654321,
  "paykey": "PK123456",
  "niNumber": "QQ123456C",
  "taxCode": "1257L",
  "previousPay": 0,
  "previousTax": 0,
  "effectiveDate": "2025-04-01",
  "sourceFileName": "p45_input_20250401.csv",
  "rowNumber": 12
}
```


API versioning and content negotiation

The API design follows the LV standard:

- stable URI space
- content negotiation through the Accept header
- no path-based versioning
- Vary: Accept returned on responses

Required media types:

Service	Media Type
clanad-policy-api	application/vnd.lv.clanad-policy.v1+json

Example headers:

```
Accept: application/vnd.lv.clanad-policy.v1+json
Authorization: Bearer <token>
```

```
Vary: Accept
```

UI

There is no evidence that a parity frontend is required for p6p9.

Ground-truth position:

- Legacy p6p9 is a console batch.

- modernization does not require a dedicated web UI for parity

If an operational support page is added later, it should be treated as an enhancement, not a parity requirement.

Non-Prod

I. Private End Point

This section proposes a P6P9-specific non-prod file-share implementation for inbound payroll files and outbound success, failure, and archive files.

Item	Value
Environment	non-prod
Resource Group	rg-p6p9-nonprod-001
Subscription	shared-aks-non-prod-001
AKS Cluster	TBD
Namespace	p6p9-nonprod
Region	UK South
Storage Account	TBD
File Share	TBD
VNet	vnet-aks-non-prod-uksouth-001
Subnet	snet-aks-non-prod-shared-uksouth-001
Private Endpoint Name	TBD
Private Endpoint IP	TBD
Resource Type	Microsoft.Storage/storageAccounts
Target Sub-Resource	file
Initial File Share Size	20 GB
Private Endpoint	enabled
Public Access	disabled

Private DNS Zone	privatelink.file.core.windows.net

Non-prod connectivity rules:

- storage reachable only through private networking
- AKS workload reaches storage, database, and Key Vault privately
- no public file-share exposure
- file share contains input, archive success and failure folders

1. Tags

Proposed non-prod mandatory tags:

Tag Name	Value
application_name	p6p9
cost_center	TBD
owner	TBD
environment	non-prod
service_class	TBD
business_unit	TBD
cohesitybackup	yes

II. User Access

Proposed non-prod file-share access:

PV Group	Access Level
PV-RS-P6P9-NonProd-Upload	Storage File Data SMB Share Contributor
PV-RS-P6P9-NonProd-Operations	Storage File Data SMB Share Contributor
PV-RS-P6P9-NonProd-Developers	Storage File Data SMB Share Contributor
PV-RS-Environment Management	Storage File Data SMB Share Contributor
PV-RS-Solution Management	Storage File Data SMB Share Contributor

III. Service Principal

Proposed non-prod batch identity:

Item	Value
Display Name	sp-azad-p6p9-nprod
Client ID	To be generated
Tenant ID	To be assigned
Access	Storage File Data SMB Share Contributor
Secret Storage	Azure Key Vault
Runtime Note	Managed identity remains preferred for AKS; service principal is retained here because the LLD requires an explicit service-principal definition

2. Prod

2.1 Private End Point

This section proposes a P6P9-specific prod file-share implementation for inbound payroll files and outbound success, failure, and archive files.

Item	Value
Environment	prod
Resource Group	rg-p6p9-prod-001
Subscription	shared-aks-prod-001
AKS Cluster	TBD
Namespace	p6p9-prod
Region	UK South
Storage Account	TBD
File Share	TBD

VNet	vnet-aks-prod-uksouth-001
Subnet	snet-aks-prod-shared-uksouth-001
Private Endpoint Name	TBD
Private Endpoint IP	TBD
Resource Type	Microsoft.Storage/storageAccounts
Target Sub-Resource	file
Initial File Share Size	20 GB
Private Endpoint	enabled
Public Access	disabled
Private DNS Zone	privatelink.file.core.windows.net

Prod connectivity rules:

- storage reachable only through private networking
- no public file-share exposure
- batch and service traffic stays inside approved private network boundaries
- file share contains input, archive success and failure folders

2.2 🔗Tags

Proposed prod mandatory tags:

Tag Name	Value
application_name	p6p9
cost_center	TBD
owner	TBD
environment	prod
service_class	TBD
business_unit	TBD

cohesitybackup	yes

2.3 Service Principal

Proposed prod batch identity:

Item	Value
Display Name	sp-azad-p6p9-prod
Client ID	To be generated
Tenant ID	To be assigned
Secret Value	Stored in Azure Key Vault
Access	Storage File Data SMB Share Contributor

2.4 User Access

Proposed prod file-share access:

PV Group	Access Level
PV-RS-P6P9-Prod-Upload	Storage File Data SMB Share Contributor
PV-RS-P6P9-Prod-Operations	Storage File Data SMB Share Contributor
PV-RS-Environment Management	Storage File Data SMB Share Contributor
PV-RS-Solution Management	Storage File Data SMB Share Contributor

Github & Pipeline

The exact pipeline names are not present in the current source pack.
Ground-truth design expectation:

- one pipeline for p6p9-batch
- one pipeline for clanad-policy-api

- one deployment manifest per deployable

Pipeline names remain TBD until confirmed.

PV Groups

Recommended design position:

- separate non-prod and prod upload groups
- separate operational access groups
- least-privilege membership

Final PV group names remain TBD.

Entra AD Registration

Required design intent:

- Entra-backed identity for service authentication
- managed identity preferred for AKS workloads
- service principal only where platform rules require it
- secrets must not be passed on command line as in the legacy batch

Final registration names and identifiers remain TBD.

Data Mapping and Schema Design

Legacy Asset	Legacy Purpose	Target Schema	Target Design
MASTER_ARCHIVE	Policy lookup and eligibility	policy_service	Accessed via **clanad-policy-api** (no direct DB access)
UPDATE_TAXCODE	Tax update execution	policy_service	Replaced by **clanad- policy -api**
P45_RECEIVED	Same tax-year adjustment	policy_service	Returned in Policy API response for processing logic
STATUS	Policy state validation	policy_service	Used by Policy API to enforce eligibility rules (e.g. L, P)
Success / Failure Files	Operational output	Azure File Share	Written to /archive/success and /archive/error
Source File Archive	Recoverability and audit	Azure File Share	Input file archived or moved based on processing outcome
Batch Run State	Run-level tracking	policy_service	Managed via Spring Batch metadata tables
Batch File State	File-level processing state	policy_service	Managed implicitly via job execution and file lifecycle
Batch Row Outcome	Row-level tracking	Azure File Share	Captured in success/error output files (not DB tables)

Proposed database objects

Schema	Object	Purpose
Policy_service	policy_tax_code_eligibility	Policy lookup source for NI, payroll number, point-one handling, status, and P45_RECEIVED logic
Policy_service	tax_code_update_audit	Audit record for every accepted update

Spring Batch Tables (runtime schema)

Table	Role
BATCH_JOB_INSTANCE	One row per logical job run identity: JOB_NAME, JOB_KEY (parameters fingerprint), VERSION. Unique on (JOB_NAME, JOB_KEY). Primary key JOB_INSTANCE_ID.
BATCH_JOB_EXECUTION	One row per **attempt** to run that job instance: CREATE_TIME, START_TIME, END_TIME, STATUS, EXIT_CODE, EXIT_MESSAGE, LAST_UPDATED, VERSION. FK to BATCH_JOB_INSTANCE.
BATCH_JOB_EXECUTION_PARAMS	Job parameters for each execution: PARAMETER_NAME, PARAMETER_TYPE, PARAMETER_VALUE, IDENTIFYING (whether the param participates in job identity). FK to BATCH_JOB_EXECUTION.
BATCH_STEP_EXECUTION	Per-step stats: STEP_NAME, times, STATUS, COMMIT_COUNT, READ_COUNT, FILTER_COUNT, WRITE_COUNT, skip counts (READ_SKIP_COUNT, WRITE_SKIP_COUNT, PROCESS_SKIP_COUNT), ROLLBACK_COUNT, EXIT_CODE / EXIT_MESSAGE, LAST_UPDATED. FK to BATCH_JOB_EXECUTION.
BATCH_STEP_EXECUTION_CONTEXT	Serialized step execution context (restart/checkpoint data): SHORT_CONTEXT, SERIALIZED_CONTEXT. PK STEP_EXECUTION_ID → BATCH_STEP_EXECUTION.
BATCH_JOB_EXECUTION_CONTEXT	Serialized job-level execution context. PK JOB_EXECUTION_ID → BATCH_JOB_EXECUTION.

Rollback Plan

The application uses **Spring Batch chunk-based transactions** with chunk-size = 100(size is taken as example will changed based on requirement).

- Each chunk of 100 records is **one PostgreSQL transaction**.
- All three DB operations — pay_header UPDATE, pafilnot INSERT, master_archive UPDATE — execute inside that single transaction.
- If anything fails within a chunk, **PostgreSQL rolls back all three operations atomically**.
- A chunk that already committed **cannot be automatically rolled back** — manual SQL reversal is required.

Rollback scenarios

Scenario	Rollback Strategy
Row-level business validation failure	Record is skipped and written to failure file; no rollback required for other records.
API failure (transient –	Retry using configured retry policy (exponential backoff); if retries fail, record is marked

retryable)	as failed and written to error output.
API failure (non-retryable – 4xx)	Record is rejected and written to failure file; processing continues for remaining records.
Chunk-level failure	Current chunk transaction is rolled back; previously committed chunks remain unaffected.
Job failure (technical error)	Job stops with FAILED status; input file is moved to error archive for investigation.
Job restart	Job can be restarted using same file; processing resumes from last successful chunk (Spring Batch remembers the last committed chunk position in its meta-tables and resumes from record 101 on the next run).
Incorrect data update (downstream)	Requires manual or service-level correction via Tax API; P6P9 does not perform compensating rollback of external updates.

Recovery handling

Scenario	Recovery Approach
Row-level validation failure	Record is skipped and written to failure file; no recovery required as processing continues.
Transient API failure (5xx / network)	Automatically retried using retry policy; if still failing, record is written to failure output.
Non-retryable API failure (4xx)	Record is rejected and captured in failure file; no retry attempted.
Job failure (technical error)	Job marked as FAILED; input file moved to error archive for investigation and reprocessing.
Partial processing (some records failed)	Successful records remain committed; failed records can be corrected and reprocessed via new input file.
Job restart	Restart supported using same job parameters (filePath); resumes from last successful chunk using Spring Batch metadata.
File reprocessing	Failed or corrected files can be reintroduced into input folder for re-execution.
Downstream update issues	Recovery handled via Tax API or operational support; P6P9 does not perform compensating updates.

Monitoring and Logging

Logging

Captured events:

- batch start and end
- file processing
- row validation errors
- service failures
- retry attempts

Metrics

Metric	Description

batch_runs_total	Total batch executions
rows_processed_total	Rows processed
rows_failed_total	Failed rows
batch_duration	Execution time

Alerts

Condition	Action
Batch job failure	Alert operations/support team
High failure rate (e.g. > 5%)	Alert support team for investigation
Repeated API failures / service unavailable	Alert platform/support team
Long-running job (threshold breach)	Alert operations team

Validation Checkpoints

1. File intake, validation, tax-code update, and success or failure output remain functionally equivalent to the legacy process.
2. P6P9 operates strictly as a **batch job**, not as a user-facing or always-on application.
3. Policy eligibility logic is owned and exposed via **clanad-policy-api**.
4. Tax-code update logic is owned and executed via **clanad-policy-api**.
5. API versioning is implemented using **Accept header content negotiation**.
6. File storage (Azure File Share) and related resources remain **private and secured** (non-public access).
7. Azure identities, RBAC roles, and resource configurations are defined and validated prior to deployment.

Final Design Position

The correct modernization position for p6p9, based on the current evidence, is:

- a private AKS-hosted batch capability
- Azure File Share for controlled file intake and result output
- policy validation and tax update through clanad-policy-api
- content-negotiated API versioning
- no mandatory frontend for parity